Experiment # 13

Implementation of Multi-Threading in Linux (Simultaneous execution of two or more threads)

Objective

In this lab we will learn about execution of threads.

Software Tools

- Ubuntu 16.04
- GCC Complier

Theory

Overview of Thread:

A thread is a single sequence stream within a process. Threads are executed within a process. They are sometimes called lightweight processes. In a process, threads allow multiple executions of streams.

What are the differences between process and thread?

Threads are not independent of one other like process as a result threads shares with other threads their code section, data section and OS resources like open files and signals. But, like process, a thread has its own program counter (PC), a register set, and a stack space.

Why Multithreading?

Threads are popular way to improve application through parallelism. For example, in a browser, multiple tabs can be different threads. MS word uses multiple threads, one thread to format the text, other thread to process inputs, etc.

Threads operate faster than processes due to following reasons:

1) Thread creation is much faster.
2) Context switching between threads is much faster.
3) Threads can be terminated easily
4) Communication between threads is faster.

Thread Creation

Normally when a program starts up and becomes a process, it starts with a default thread. So we can say that every process has at least one thread of control. A process can create extra threads using the following function:

```
#include <pthread.h>

int pthread_create(pthread_t *restrict tidp, constpthread_attr_t *restrict attr, void *(*start_rtn)(void), void *restrict arg)
```

The above function requires four arguments, lets first discuss a bit on them :

thread_create : It is used to create a new thread.

Syntax: Int pthread_create(pthread_t *thread, pthread_attr_t *attr, void *(* start_routine) (void *), void *arg)

Pthread_t: It is defining a thread pointer. When a thread is created identifier is written into the variable to which the pointer points. This identifier helps to refer to thread.

Pthread_attr_t: It is used to set the thread attributes. Usually there is no need to set the thread attributes. Pass this argument as NULL.

Name of function: The name of the function to be started by the thread for execution.

Arguments to be passed to the function: When a new thread is created it executes the function pointed by the function variable name.

- The first argument is a pthread_t type address. Once the function is called successfully, the variable whose address is passed as first argument will hold the thread ID of the newly created thread.
- The second argument may contain certain attributes which we want the new thread to contain. It could be priority etc.
- The third argument is a function pointer. This is something to keep in mind that each thread starts with a function and that functions address is passed here as the third argument so that the kernel knows which function to start the thread from.
- As the function (whose address is passed in the third argument above) may accept some arguments also so we can pass these arguments in form of a pointer to a void type. Now, why a void type was chosen? This was because if a function accepts more than one argument then this pointer could be a pointer to a structure that may contain these arguments.

Example1.c

```
#include<stdio.h>
#include<string.h>
#include<pthread.h>
#include<stdlib.h>
#include<unistd.h>
```

```
pthread_t tid[2];

void* doSomeThing(void *arg)
{
 unsigned long i = 0;
 pthread_t id = pthread_self();

 if(pthread_equal(id,tid[0]))
    {
     printf("\n First thread processing\n");
     printf("%d\n",pthread_self());
    }
else
    {
     printf("\n Second thread processing\n");
     printf("%d\n",pthread_self());
    }

for(i=0; i<(0xFFFFFFFF);i++);

}

int main(void)
{ int i = 0;
   int err;
   while(i < 2)
    {
     err = pthread_create(&(tid[i]), NULL, &doSomeThing, NULL);

     if (err != 0)
          printf("\ncan't create thread :[%s]", strerror(err));
     else
          printf("\n Threads created successfully\n");

     i++;

     pthread_join(tid[i], NULL);
     sleep(5);
    }
return 0;
}
```

For compilation of the code use gcc -o Example1.c Example1 –lpthread

So what this code does is :

- It uses the pthread_create() function to create two threads
- The starting function for both the threads is kept same.
- Inside the function 'doSomeThing()', the thread uses pthread_self() and pthread_equal() functions to identify whether the executing thread is the first one or the second one as created.
- Also, Inside the same function 'doSomeThing()' a for loop is run so as to simulate some time consuming work.

C Thread Termination Example

Let's take an example where we use the above discussed functions:

Example2.c

```
#include<stdio.h>
#include<string.h>
#include<pthread.h>
#include<stdlib.h>
#include<unistd.h>

pthread_ttid[2];
int ret1,ret2;

void* doSomeThing(void *arg)
{
    unsigned long i = 0;
    pthread_t id = pthread_self();

    //for(i=0; i<(0xFFFFFFFF);i++);

    if(pthread_equal(id,tid[0]))
    {
        printf("\n First thread processing done\n");
        ret1 = 100;
        pthread_exit(&ret1);
    }
```

```
    else
    {
   printf("\n Second thread processing done\n");
        ret2 = 200;
        pthread_exit(&ret2);
   }

   return NULL;
}

int main(void)
{ int i = 0;
   int err;
   int *ptr[2];
   while(i < 2)
   {
        err = pthread_create(&(tid[i]), NULL, &doSomeThing, NULL);

        if (err != 0)
            printf("\ncan't create thread :[%s]", strerror(err));
        else
            printf("\n Thread created successfully\n");

        i++;
    }

    pthread_join(tid[0], (void**)&(ptr[0]));
    pthread_join(tid[1], (void**)&(ptr[1]));

    printf("\n return value from first thread is [%d]\n", *ptr[0]);
    printf("\n return value from second thread is [%d]\n", *ptr[1]);

    return 0;
}
```

In the code above:

- We created two threads using pthread_create()
- The start function for both the threads is same i.e. doSomeThing()
- The threads exit from the start function using the pthread_exit() function with a return value.

- In the main function after the threads are created, the pthread_join() functions are called to wait for the two threads to complete.
- Once both the threads are complete, their return value is accessed by the second argument in the pthread_join() call.

Lab Task:

- **Write a program for matrix addition, subtraction and multiplication using multithreading.**

Important note: Lab Report must be according to the Lab Report Format available on Google Classroom and must be submitted on time. Two similar reports will get zero marks. So, don't try to copy your fellow reports.